

Guía Práctica: Evaluación y Validación de Modelos en Minería de Datos

1. Introducción y Contextualización

En el campo de la Minería de Datos, el objetivo central es la generación de modelos o hipótesis a partir de la evidencia contenida en un conjunto de datos. Sin embargo, la construcción de un modelo es solo la mitad del camino; como expertos, nuestra responsabilidad ética y técnica radica en determinar si dicho modelo posee la validez necesaria para ser aplicado en entornos reales.

La etapa de evaluación es, por tanto, crucial. Debemos medir la calidad de nuestras hipótesis con la mayor exactitud posible. En disciplinas de alta criticidad, como el diagnóstico médico, un error de predicción puede traducirse en consecuencias vitales. Por ejemplo, en la detección de células cancerígenas, no basta con "creer" que el modelo funciona; es imperativo conocer su nivel de precisión exacto para garantizar que el conocimiento extraído sea fiable y, sobre todo, generalizable ante datos nunca antes vistos.

2. Objetivos

A través de este ejercicio práctico, el estudiante desarrollará las siguientes competencias clave:

- Diagnosticar fenómenos de ajuste: Identificar cuándo un modelo padece de overfitting (sobreajuste) o underfitting (bajo ajuste).
- Implementar divisiones de datos robustas: Aplicar técnicas de Train/Test Split integrando estratificación para preservar la representatividad estadística.
- Asegurar la generalización: Utilizar Validación Cruzada (K-Fold Cross Validation) como estándar de oro para la validación de modelos.
- Ejecutar optimización sistemática: Aplicar búsquedas de hiperparámetros mediante GridSearchCV para encontrar la configuración óptima del modelo.

3. Fundamentos Teóricos: El Problema del Ajuste y la Generalización

El objetivo principal de entrenar un modelo de Machine Learning no es que memorice los datos del pasado, sino que **generalice**: es decir, que sea capaz de realizar predicciones correctas ante datos nuevos que nunca antes ha visto.

En este proceso, el modelo intenta encontrar un equilibrio entre dos fuerzas opuestas: el **Sesgo (Bias)** y la **Varianza (Variance)**. Cuando este equilibrio se rompe, surgen los dos grandes problemas del ajuste: Underfitting y Overfitting.

3.1 Underfitting (Bajo ajuste)

Ocurre cuando el modelo es **demasiado simple** para capturar la estructura subyacente de los datos.

- **Síntoma:** El modelo se equivoca mucho tanto en el entrenamiento como en las pruebas (alto sesgo).
- **Analogía:** Es como intentar estudiar para un examen de cálculo memorizando solo las tablas de multiplicar; el modelo no tiene la capacidad de entender la complejidad del problema.

3.2 Overfitting (Sobreajuste)

Ocurre cuando el modelo es **demasiado complejo** y, en lugar de aprender el patrón general, "memoriza" el ruido y los detalles específicos del conjunto de entrenamiento.

- **Síntoma:** El modelo tiene un rendimiento casi perfecto con los datos de entrenamiento, pero falla drásticamente al enfrentarse a datos nuevos (alta varianza).
- **Analogía:** Es como memorizar las respuestas exactas de un examen de práctica; si en el examen real cambian una sola pregunta, no sabrás cómo responder.

Error Crítico y Optimismo Excesivo: Un error recurrente en la práctica académica es evaluar la calidad final del clasificador utilizando el mismo conjunto de datos con el que fue entrenado. Esta práctica conduce a un "sesgo de optimismo" o *Data Leakage*, donde los resultados parecen excelentes simplemente porque el modelo ya conoce las respuestas. Para una evaluación legítima, el modelo debe ser desafiado con datos "invisibles" durante su fase de aprendizaje.

4. Fase 1: Preparación del Entorno y Análisis Exploratorio (EDA)

Como primer paso, configuramos nuestro entorno de trabajo con las librerías estándar para la manipulación de datos, visualización y aprendizaje automatizado.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
plt.style.use('ggplot')
```

El Set de Datos: "Diabetes de los indios Pima"

Este dataset nos permite predecir la presencia de diabetes basándonos en registros médicos. Consta de un DataFrame de **768 filas y 9 columnas**.

Variables Independientes (Features):

1. Pregnancies (Embarazos)
2. Glucose (Glucosa)
3. BloodPressure (Presión arterial)
4. SkinThickness (Grosor de pliegue cutáneo)
5. Insulin (Insulina)
6. BMI (Índice de masa corporal)
7. DiabetesPedigreeFunction (Función de pedigrí)
8. Age (Edad)

Variable Objetivo (Label):

- Outcome: (1 = Presencia de diabetes, 0 = Ausencia).

Carga de datos y segmentación inicial:

```
#Cargamos nuestro set de datos
df = pd.read_csv('diabetes.csv')
```

```
#Separamos nuestros datos en características (X) y etiqueta (y)
X = df.drop('Outcome', axis=1).values
y = df['Outcome'].values
```

5. Fase 2: División de Datos y Entrenamiento Inicial

La técnica Train/Test Split separa los datos para que el modelo sea capaz de entrenar un grupo y sea evaluado en otro grupo desconocido.

Configuración Experimental:

- **Tamaño de prueba (Test size):** 40%, para asegurar un volumen de validación suficiente.
- **Semilla de aleatoriedad (Random State):** 42, para garantizar la reproducibilidad.
- **Estratificación (stratify=y):** Esta es una práctica esencial en conjuntos de datos médicos donde la prevalencia de la enfermedad puede estar desbalanceada. La estratificación garantiza que la proporción de personas con y sin diabetes sea la misma en ambos conjuntos (entrenamiento y prueba), evitando sesgos en la evaluación.

```
#División de datos con estratificación
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=42, stratify=y
)
```

```
#A continuación, instanciamos y entrenamos un modelo k-NN inicial con k=7
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(X_train, y_train)
```

6. Fase 3: Diagnóstico de Overfitting y Underfitting

Para identificar el ajuste ideal, variamos el número de vecinos (k) de 1 a 9. Si graficáramos estos resultados, observaríamos un comportamiento revelador:

1. **Zona de Overfitting (k=1):** Con un solo vecino, la precisión en el entrenamiento es del 100% (el modelo se ajusta a cada punto individual), pero la precisión en el conjunto de prueba es significativamente baja, pues el modelo no ha aprendido patrones, sino que ha memorizado el ruido.
2. **Punto de Equilibrio (k=7):** A medida que aumentamos k, la precisión de entrenamiento disminuye gradualmente mientras que la de prueba sube, alcanzando su **máximo en k=7**.
3. **Zona de Underfitting:** Si seguimos aumentando k más allá del punto óptimo, ambas precisiones comienzan a decaer, ya que el modelo se vuelve demasiado generalista y pierde la capacidad de distinguir patrones locales importantes.

```
#importamos el clasificador
from sklearn.neighbors import KNeighborsClassifier

#Configuramos matrices para almacenar la precisión de las pruebas y el entrenamiento
neighbors = np.arange(1,9)
train_accuracy = np.empty(len(neighbors))
test_accuracy = np.empty(len(neighbors))

for i,k in enumerate(neighbors):

    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)

    #Calculamos la exactitud con entrenamiento
    train_accuracy[i] = knn.score(X_train, y_train)

    #Calculamos la exactitud con test
    test_accuracy[i] = knn.score(X_test, y_test)
```

```
#Generamos un gráfico con las iteraciones con diferentes K vs la exactitud
plt.title('k-NN Varying number of neighbors')
plt.plot(neighbors, test_accuracy, label='Testing Accuracy')
plt.plot(neighbors, train_accuracy, label='Training accuracy')
plt.legend()
plt.xlabel('Number of neighbors')
plt.ylabel('Accuracy')
plt.show()
```

7. Fase 4: Evaluación Detallada del Clasificador

Utilizando el valor óptimo identificado manualmente (**k=7**), obtenemos los siguientes resultados de rendimiento:

```
#A continuación, instanciamos y entrenamos un modelo k-NN inicial con k=7
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(X_train, y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier(n_neighbors=7)
```

```
#Generamos el valor del SCORE del modelo
knn.score(X_test, y_test)
```

```
0.7305194805194806
```

- **Precisión Global (Score):** 0.7305 (73%).

7.1 Matriz de Confusión

Es la herramienta definitiva para observar cómo se distribuyen las predicciones reales frente a las esperadas:

```
from sklearn.metrics import confusion_matrix
```

```
#realizamos predicciones
y_pred = knn.predict(X_test)
```

```
confusion_matrix(y_test, y_pred)
```

```
array([[165,  36],
       [ 47,  60]])
```

7.2 Informe de Clasificación (Classification Report)

El **Informe de Clasificación** es una de las herramientas de evaluación más completas en problemas de clasificación (binaria o multiclase). A diferencia de la precisión global (*accuracy*), que puede ser engañosa si las clases están desbalanceadas, este informe desglosa el rendimiento del modelo **para cada clase individual** a través de tres métricas fundamentales y una medida de balance:

7.3 Las Métricas Clave

Para entender este informe, primero debemos comprender qué evalúa cada columna:

Precisión (Precision): De todos los pacientes que el modelo predijo que tenían diabetes, ¿cuántos realmente la padecen?

$$\text{Fórmula: Recall} = \frac{\text{Diabéticos Reales Detectados}}{\text{Total de Diabéticos Reales en el grupo}}$$

- **El escenario (Falso Positivo):** Un Falso Positivo ocurre cuando el modelo le dice a un paciente **sano** que tiene diabetes.
- **Consecuencia:** El paciente sufrirá estrés psicológico innecesario y se le someterá a exámenes de sangre adicionales (más costosos y molestos) para confirmar el diagnóstico antes de medicarlo. Una alta precisión reduce estas "falsas alarmas".

Recuperación / Sensibilidad (Recall): De todos los pacientes que realmente tienen diabetes en el hospital, ¿cuántos logró detectar el modelo?

$$\text{Fórmula: Recall} = \frac{\text{Diabéticos Reales Detectados}}{\text{Total de Diabéticos Reales en el grupo}}$$

- **El escenario (Falso Negativo):** Un Falso Negativo ocurre cuando el modelo le dice a un paciente **diabético** que está completamente sano.
- **Consecuencia: Este es el escenario más peligroso.** El paciente se va a casa sin saber que está enfermo, no recibe tratamiento, y su condición puede empeorar gravemente con el tiempo (daño renal, problemas cardiovasculares, etc.). En medicina, casi siempre se prioriza un **Recall muy alto**, porque es preferible hacerle pruebas extra a alguien sano que dejar ir a un enfermo a casa.

Puntuación F1 (F1-Score): Es el balance ideal. Si buscas un modelo que no sature los laboratorios del hospital con exámenes de confirmación innecesarios (alta precisión), pero que al mismo tiempo no deje escapar a ningún paciente enfermo a casa (alto recall), el F1-Score será tu métrica de referencia.

```
#reporte de clasificación
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test,y_pred))
```

	precision	recall	f1-score	support
0	0.78	0.82	0.80	201
1	0.62	0.56	0.59	107
accuracy			0.73	308
macro avg	0.70	0.69	0.70	308
weighted avg	0.73	0.73	0.73	308

8. Fase 5: Validación Cruzada y Optimización de Hiperparámetros

En las etapas previas evaluamos el modelo utilizando una única división de datos (*Train/Test Split*). Sin embargo, esto plantea un riesgo metodológico: el azar. Dependiendo de cómo se hayan distribuido los registros de forma aleatoria, podríamos obtener un resultado inusualmente "bueno" (optimista) o "malo" (pesimista) simplemente por suerte.

Para garantizar que el modelo realmente generalice y sea confiable al diagnosticar nuevos pacientes con diabetes, implementamos un proceso robusto en dos pasos.

8.1 Validación Cruzada de K-Veces (K-Fold Cross Validation)

La Validación Cruzada de K veces mitiga el sesgo de la selección aleatoria dividiendo el conjunto de datos de entrenamiento en K submuestras (o *folds*) de igual tamaño. El proceso se ejecuta de forma iterativa siguiendo estos principios:

1. **Rotación Sistemática:** El modelo se entrena y valida un total de K veces. En cada iteración (i), la submuestra i se reserva exclusivamente como conjunto de validación (prueba interna), mientras que las $K-1$ submuestras restantes se fusionan para formar el conjunto de entrenamiento.
2. **Uso Total de los Datos:** Al finalizar el ciclo, cada registro del dataset habrá sido utilizado exactamente una vez para validar y $K-1$ veces para entrenar.
3. **Métrica Consolidada:** En lugar de obtener una única puntuación de rendimiento, obtenemos K puntuaciones diferentes. El rendimiento final del modelo se reporta mediante dos valores:
 - **La Media (μ):** El promedio de los resultados, que representa la capacidad real esperada del modelo.
 - **La Desviación Estándar (σ):** Mide la variabilidad entre las iteraciones. Una desviación estándar baja indica que el modelo es estable y no depende del azar de los datos.

8.2 Optimización con GridSearchCV: Sintonizando el Algoritmo KNN

Cada algoritmo de Machine Learning posee parámetros internos que el desarrollador debe configurar antes de iniciar el entrenamiento, conocidos como **hiperparámetros**. En el caso del algoritmo *K-Nearest Neighbors* (KNN), el hiperparámetro crítico es `n_neighbors` (k), que define cuántos pacientes vecinos (similares) se consultarán para predecir si un nuevo paciente tiene diabetes o no.

La elección de este número altera drásticamente el comportamiento del modelo frente al problema del ajuste:

- **Un k muy bajo (ej. `n_neighbors = 1`):** El modelo se vuelve sumamente complejo y sensible al ruido. Intentará ajustarse perfectamente a cada dato de entrenamiento, lo que conduce irremediablemente a **Overfitting** (Alta Varianza).

- **Un k muy alto (ej. `n_neighbors = 50`):** El modelo se vuelve demasiado simple y "vota" basándose en la mayoría estadística global en lugar de en la similitud local. Esto genera **Underfitting** (Alto Sesgo), ignorando los patrones particulares de la diabetes.

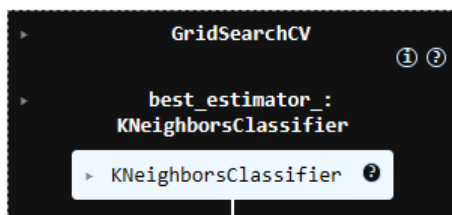
¿Cómo opera GridSearchCV en esta fase?

Para encontrar el equilibrio perfecto (el punto óptimo entre sesgo y varianza), utilizamos **GridSearchCV** (*Grid Search Cross-Validation*), que automatiza una búsqueda exhaustiva bajo la siguiente metodología:

1. **Definición de la Rejilla (*Grid*):** Creamos un rango extendido de búsqueda para `n_neighbors` desde 1 hasta 50.
2. **Evaluación Cruzada Integrada:** Para cada uno de los valores posibles en ese rango (1, 2, 3... hasta 50), GridSearchCV no realiza una prueba simple, sino que ejecuta una **Validación Cruzada completa de K -folds**.
3. **Criterio de Selección:** Evaluará cada combinación utilizando la métrica de rendimiento seleccionada para el proyecto (por ejemplo, optimizando el *Recall* o el *F1-Score* de la clase de pacientes diabéticos).
4. **Resultado Óptimo:** La técnica selecciona automáticamente el valor de `n_neighbors` que obtuvo el mejor promedio en la validación cruzada y, finalmente, reentrena el modelo utilizando esa configuración ideal con todo el conjunto de datos de entrenamiento.

```
from sklearn.model_selection import GridSearchCV

param_grid = {'n_neighbors': np.arange(1, 50)}
knn_base = KNeighborsClassifier()
# Aplicamos 4 pliegues de validación cruzada
knn_cv = GridSearchCV(knn_base, param_grid, cv=4)
knn_cv.fit(X, y)
```



Hallazgos Finales de la Optimización:

- **Mejor parámetro encontrado:** `n_neighbors`: 17.
- **Mejor precisión (Best Score):** 0.7473 (Aprox. 75%).

```
▶ knn_cv.best_score_  
... np.float64(0.7473958333333334)
```

Conclusión Técnica: Mientras que la búsqueda manual nos sugirió $k=7$, la validación cruzada —que es el estándar de oro— reveló que **$k=17$** es un valor más robusto. Esta diferencia demuestra que el proceso de optimización sistemática es superior para encontrar modelos con verdadera capacidad de generalización

9. Conclusiones y Preguntas de Reflexión

Como futuros científicos de datos, deben sintetizar estos hallazgos no solo desde el código, sino desde el impacto del modelo:

1. **Sobre la Robustez:** ¿Por qué un modelo con $k=17$ es preferible a uno con $k=7$, aunque la mejora numérica parezca pequeña (del 73% al 75%)? Considere la diferencia entre un resultado obtenido por azar en una sola división frente a la estabilidad de la validación cruzada.
2. **Sobre la Complejidad:** Si observamos una precisión de entrenamiento del 99% pero una precisión de prueba del 60%, ¿qué nos dice esto sobre la complejidad del modelo y su utilidad práctica en el mundo real?
3. **Sobre el Dominio Médico:** En el contexto de la diabetes, tenemos **47 falsos negativos**. ¿Qué implicaciones éticas y de salud tiene que el modelo clasifique como "sanos" a 47 pacientes que realmente padecen la enfermedad? ¿Es la precisión global la métrica más importante en este caso?

```
knn_cv.best_params_  
{'n_neighbors': np.int64(17)}
```